

Supporting Synchronous and Asynchronous Agent Execution

Architecture options, target design, migration plan, scalability, multi-shard strategy and cloud-native operating model

Architecture Decision and Target-State Design

Document date: 5 August 2026

Source baseline: Current asynchronous architecture and architecture decision document dated 4 August 2026

Scope: Agentic Orchestration Service, Kafka execution backbone, Executor Service, PostgreSQL workflow state and caller delivery channels

Primary objective: Support synchronous, asynchronous and streaming callers through one scalable execution engine

Target principle: True orchestration in Orchestration Service; durable one-step execution through Kafka; delivery mode separated from execution semantics

Recommended decision

Adopt a single durable workflow engine. The Orchestration Service owns the plan and every workflow transition. Executor pods execute one independently retryable step. Kafka remains the durable boundary. Sync, async and streaming endpoints differ only in how the final or intermediate results are delivered.

Contents

1. Executive summary
2. Scope, assumptions and architecture principles
3. Current architecture: asynchronous and choreographed
4. Option A: separate fully synchronous execution path plus existing async path
5. Option B: hybrid endpoints over the existing Kafka execution path
6. Option C: hybrid endpoints with durable result-store synchronization
7. Recommended target: true orchestration with durable Kafka steps
8. Comparative feature and trade-off analysis
9. Detailed target-state component design
10. API and delivery-mode design
11. Workflow state, Kafka contracts and reliability controls
12. Migration from the current asynchronous architecture
13. Scalability and performance model
14. Multi-shard and execution-cell architecture
15. Cloud-native deployment and operations
16. Failure scenarios and recovery behavior
17. Risks, constraints and decision gates
18. Final recommendation and end state

1. Executive summary

The current platform is asynchronous at the API boundary and choreographed internally. Orchestration creates the plan and publishes it to Kafka, but the Executor advances the routing slip, republishes the next step and determines when the plan is complete. The caller receives an immediate acknowledgement and must own a Kafka response topic or webhook to receive the final result.

Four architecture shapes have been evaluated: the current async model; a separate fully synchronous path using direct service calls; a hybrid sync/async façade that keeps the current Kafka execution engine; and the recommended target in which Orchestration becomes a durable workflow engine while Kafka remains the execution transport.

Key conclusion

Making Orchestration a real orchestrator is a better long-term design. Making the whole internal path synchronous is not required and would weaken durability, backpressure and pod independence. The strongest design combines true orchestration with durable asynchronous step transport and separate caller-facing delivery modes.

- Near-term capability: add sync endpoints without changing execution semantics by waiting on a durable final result.
- Strategic target: move next-step decisions, retries, cancellation, progress and finalization into Orchestration.
- Executor target: consume one command, execute one step, emit one result and stop.
- Scale model: independent API, orchestration-worker, executor, outbox and delivery deployments.
- Shard model: one workflow has one owning shard or execution cell; messages are keyed by `workflow_id`.

- Cloud-native requirement: no workflow depends on one pod remaining alive.

2. Scope, assumptions and architecture principles

2.1 Scope

- Existing asynchronous POST endpoints and their external behavior.
- New synchronous and optional streaming endpoints.
- Orchestration and Executor service responsibilities.
- Kafka topics, consumer groups and message contracts.
- Durable workflow state, step state, result storage and delivery registration.
- Horizontal scaling, shard ownership, failure recovery and cloud-native deployment.

2.2 Baseline facts from the current design

- Orchestration and Executor are FastAPI services and both are deployed as multiple pods.
- One internal Kafka topic currently carries both AGENT_EXECUTION_REQUEST and AGENT_EXECUTION_FINAL_RESPONSE messages, keyed by `x_correlation_id`.
- StaticPlanner builds the ordered plan, but the Executor advances it through a routing-slip loop.
- The current external result paths are the caller-owned Kafka topic or caller webhook.
- PostgreSQL schema `gssp_agentic` is shared by both services.
- The OpenShift Route timeout is 900 seconds, creating a hard ceiling for held HTTP connections.
- Current Kafka consumers use `auto_offset_reset=latest` with `auto-commit`, which can leave in-flight runs orphaned after pod failure.

2.3 Architecture principles

Principle	Meaning
One execution engine	Sync, async and streaming callers must not use divergent orchestration logic.
Durable workflow ownership	Plan position, retries, deadlines and final result must survive pod replacement.
Single-step executors	Executor pods execute bounded work units and do not decide the next workflow step.
At-least-once safety	Commands and results are idempotent; duplicate delivery must not duplicate side effects.
Delivery is separate from execution	HTTP, Kafka, webhook and SSE are adapters over the same completed workflow.
Shard locality	A workflow has one durable home shard, even when many pods execute its steps.
Independent scaling	API, orchestration transitions, executor capacity and delivery channels scale separately.
Bounded synchronous waiting	Long runs return 202 and continue durably rather than holding connections indefinitely.

3. Current architecture: asynchronous and choreographed

3.1 Current flow

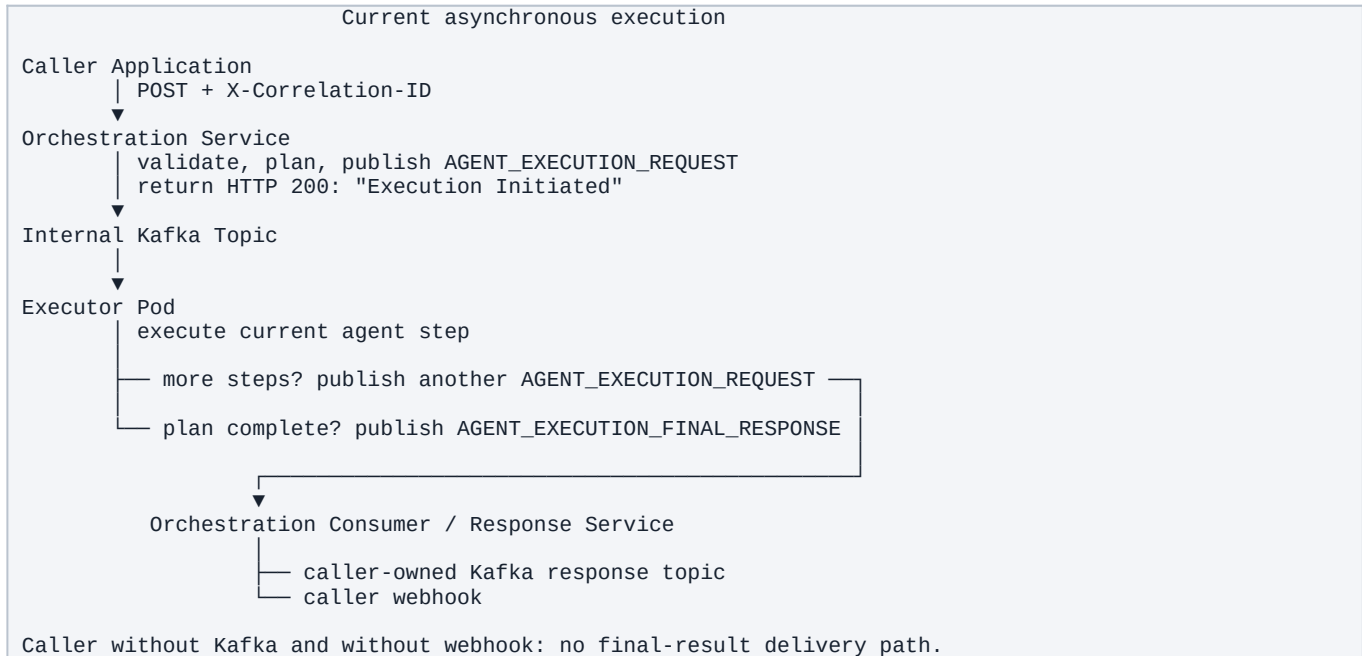


Figure 1 — Current asynchronous routing-slip flow

3.2 Current responsibility split

Capability	Current owner	Observation
Plan creation	Orchestration	StaticPlanner creates ordered steps.
Workflow advancement	Executor	Executor inspects the plan and republishes the next request.
Step execution	Executor	Agent, LLM and tool execution.
Final completion decision	Executor	Executor decides when the plan has ended.
External response delivery	Orchestration ResponseService	Caller Kafka topic or webhook.
Workflow progress ownership	No stable owner	Orchestration does not continuously hold the plan.

3.3 Strengths

- Kafka provides buffering and backpressure between request intake and executor capacity.
- Executor pods are disposable; different hops can run on different pods.
- A plan can continue even if the Orchestration API pod that accepted the request is no longer present.
- The current external async contract is simple for teams that already own Kafka or a webhook.

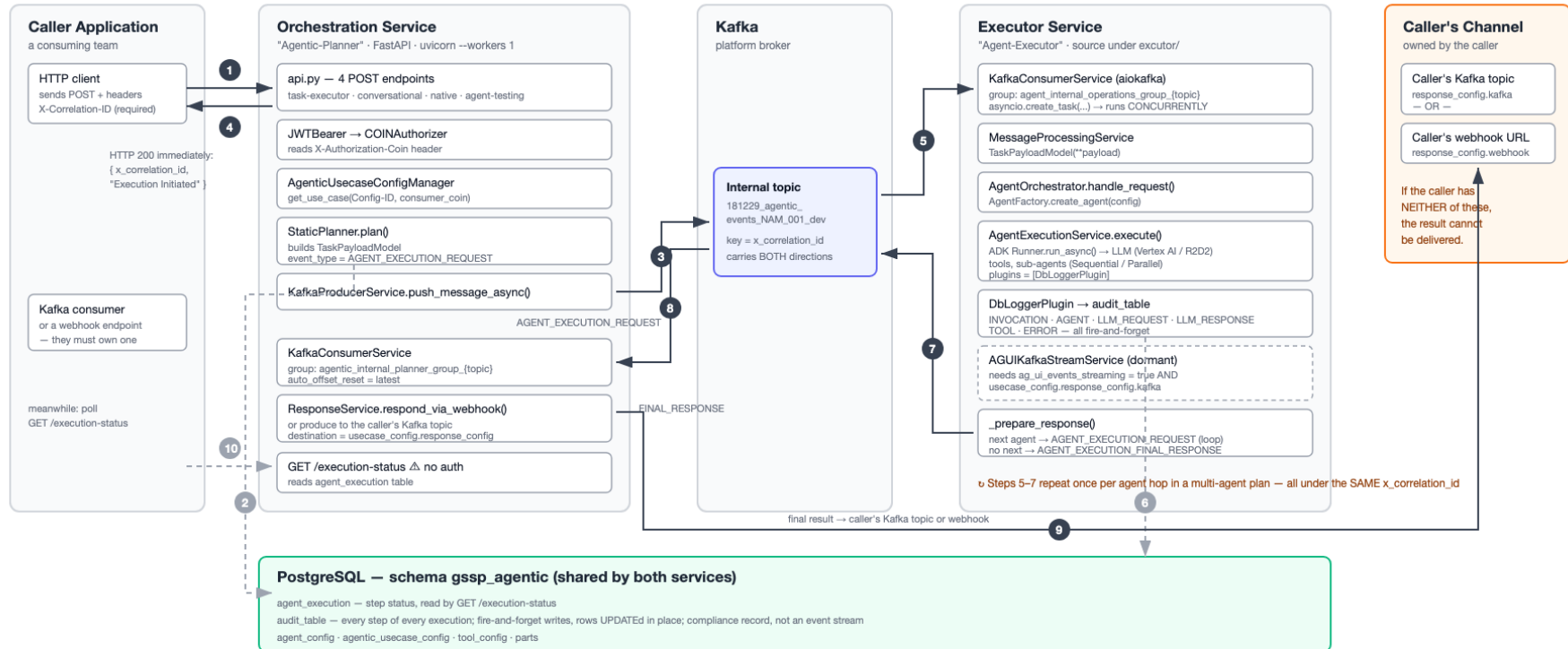
3.4 Limitations

- No final-result delivery for callers without Kafka or webhook.
- No plan-level progress because Orchestration does not own active workflow state.
- Cancellation has no stable control point.
- Retry and failure recovery are not coordinated as workflow policy.
- Current consumer offset behavior can orphan in-flight work after pod failure.
- One shared topic and shared PostgreSQL schema create future scaling and isolation concerns.

3.5 Current architecture source diagram

Current Architecture — Asynchronous Flow (as built today)

Caller must own a Kafka response topic or a webhook endpoint to receive the result. No visibility into intermediate steps.



What this design cannot do

1. **A caller with no Kafka and no webhook cannot receive the result at all.** The only delivery paths are the caller's own response topic or their webhook URL.
2. **No step visibility.** The caller sees "Execution Initiated", then silence, then a final answer. Intermediate tool calls and LLM turns are invisible unless they poll.
3. **Polling is coarse and unauthenticated.** GET /execution-status has no JWTBearer dependency — anyone who knows an x_correlation_id can read its status.
4. **AG-UI streaming exists but is unreachable.** AGUIKafkaStreamService emits rich step events — but only to the caller's own Kafka topic, so it helps only teams who already have Kafka.
5. **A pod restart orphans an execution.** Both consumers use auto_offset_reset=latest with auto-commit, so an in-flight run is never reprocessed and sits IN_PROGRESS forever.

Verified against DISCOVERY-v2.md and direct reads of agui_kafka_stream_service.py, agui_events.py, api.py, kafka_consumer_service.py and the Helm charts (2026-08-04).

Figure 2 — Current architecture supplied with the request

4. Option A: separate fully synchronous execution path plus existing async path

4.1 Architecture shape

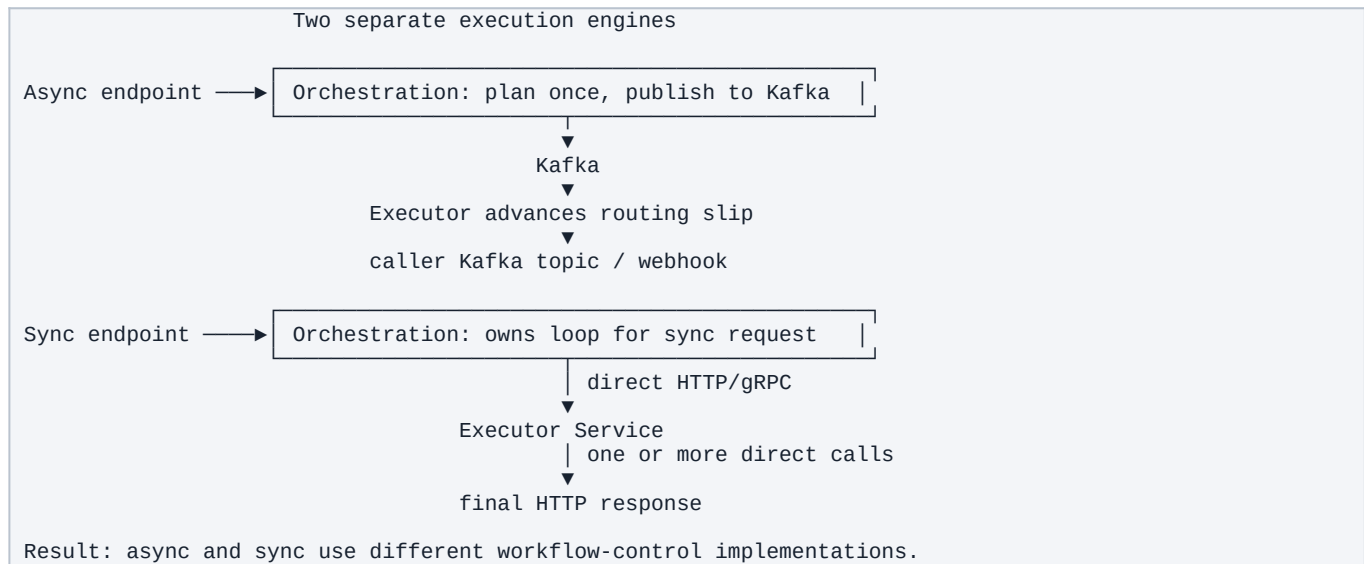


Figure 3 — Separate direct synchronous engine alongside the existing async engine

4.2 What changes

- New /sync endpoints call Executor directly through HTTP or gRPC.
- Orchestration implements the full multi-step loop for synchronous requests.
- Executor exposes a direct execution API in addition to its Kafka consumer path.
- The current async routing-slip logic remains active for existing callers.
- Separate retry, timeout, error-mapping and finalization logic is required for the direct path.

4.3 Benefits

- Simple request-response mental model for the synchronous path.
- No Kafka result fan-in for the waiting sync connection.
- Potentially lower latency for short, single-step requests.
- Orchestration can provide plan-level control for sync requests.

4.4 Trade-offs and risks

- Two execution engines will drift in sequencing, retries, auditing and error behavior.
- The Orchestration pod and selected Executor pod become coupled to the caller connection.
- Direct calls lose Kafka buffering and require admission control, circuit breakers and load shedding.
- A pod failure can lose in-memory workflow progress unless the direct path is also durably checkpointed.
- The 900-second route ceiling still applies to the caller connection.
- The design is not a low-change extension; it is a second orchestration implementation.

Assessment

Not recommended as a permanent architecture. It can be justified only for narrowly bounded, single-step, low-risk operations. It is unsuitable as the general multi-agent sync design because it duplicates the workflow engine and weakens cloud-native failure isolation.

5. Option B: hybrid endpoints over the existing Kafka execution path

5.1 Architecture shape

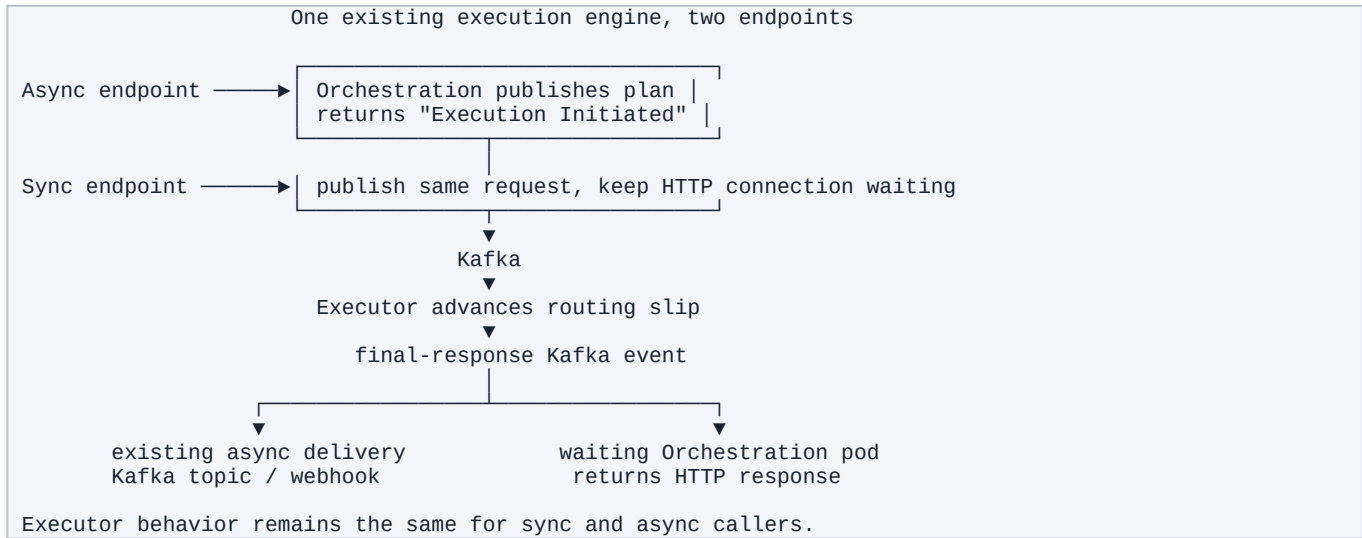


Figure 4 — Hybrid sync/async endpoints over the current routing-slip engine

5.2 Completion fan-in choices

Because several Orchestration pods may be running, the final response must reach the pod holding the synchronous connection. A normal Kafka consumer group does not guarantee that the correct pod receives the message.

- Group-less broadcast tap: every Orchestration pod reads final events and filters locally by `x_correlation_id`.
- Connection registry: the waiting pod records `x_correlation_id` to connection ownership in memory or a durable registry.
- Dedicated completion-notification layer: a result projector stores the final response and waiting pods query that durable state.

5.3 Benefits

- The Executor and multi-hop routing-slip loop remain unchanged.
- Existing async callers are unaffected.
- Kafka backpressure and executor pod independence are preserved.
- The sync capability can be delivered incrementally.

5.4 Trade-offs

- The underlying workflow remains choreographed; progress and cancellation are still weak.
- Group-less Kafka taps create read amplification as Orchestration pod count grows.
- An in-memory connection registry is lost when a pod terminates.
- A live connection still depends on one API pod remaining alive.
- The route timeout limits the maximum synchronous wait.

Assessment

Good as an incremental sync façade. It solves the caller-delivery problem with limited blast radius, but it does not solve workflow ownership, progress, cancellation or durable recovery of plan transitions.

6. Option C: hybrid endpoints with durable result-store synchronization

6.1 Architecture shape

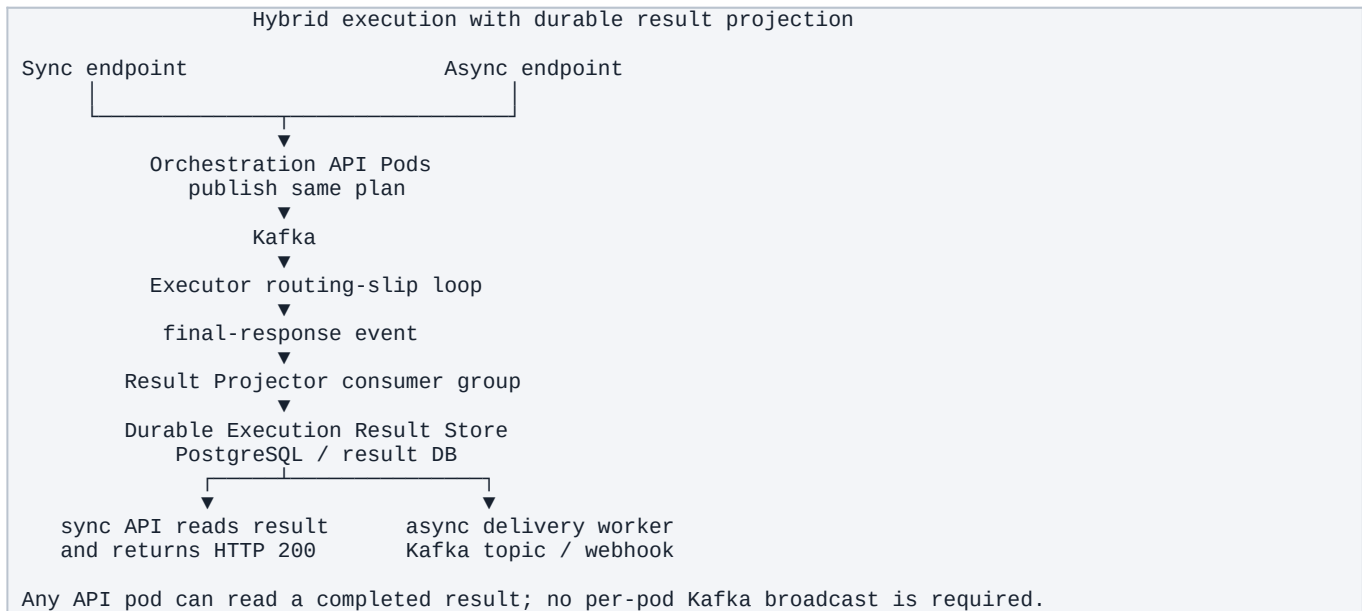


Figure 5 — Hybrid sync/async endpoints using a durable completion store

6.2 Benefits over Option B

- No group-less read amplification for blocking synchronous calls.
- No in-memory correlation registry is required for final completion.
- Any API pod can serve a retry or result lookup.
- The final response becomes durable before delivery.
- Sync timeout can return 202 while execution continues.

6.3 Remaining limitations

- The Executor still owns the routing-slip loop.
- Orchestration still lacks authoritative current-step ownership.
- Plan progress and cancellation remain limited.
- Result-store polling adds database reads proportional to active sync requests.
- It depends on durable storage of the complete final response, not status alone.

Assessment

Recommended as the safest interim design if the immediate goal is a synchronous API. It is also a useful stepping stone because the durable result store, idempotency and authenticated status APIs are required by the final architecture.

7. Recommended target: true orchestration with durable Kafka steps

7.1 Target architecture

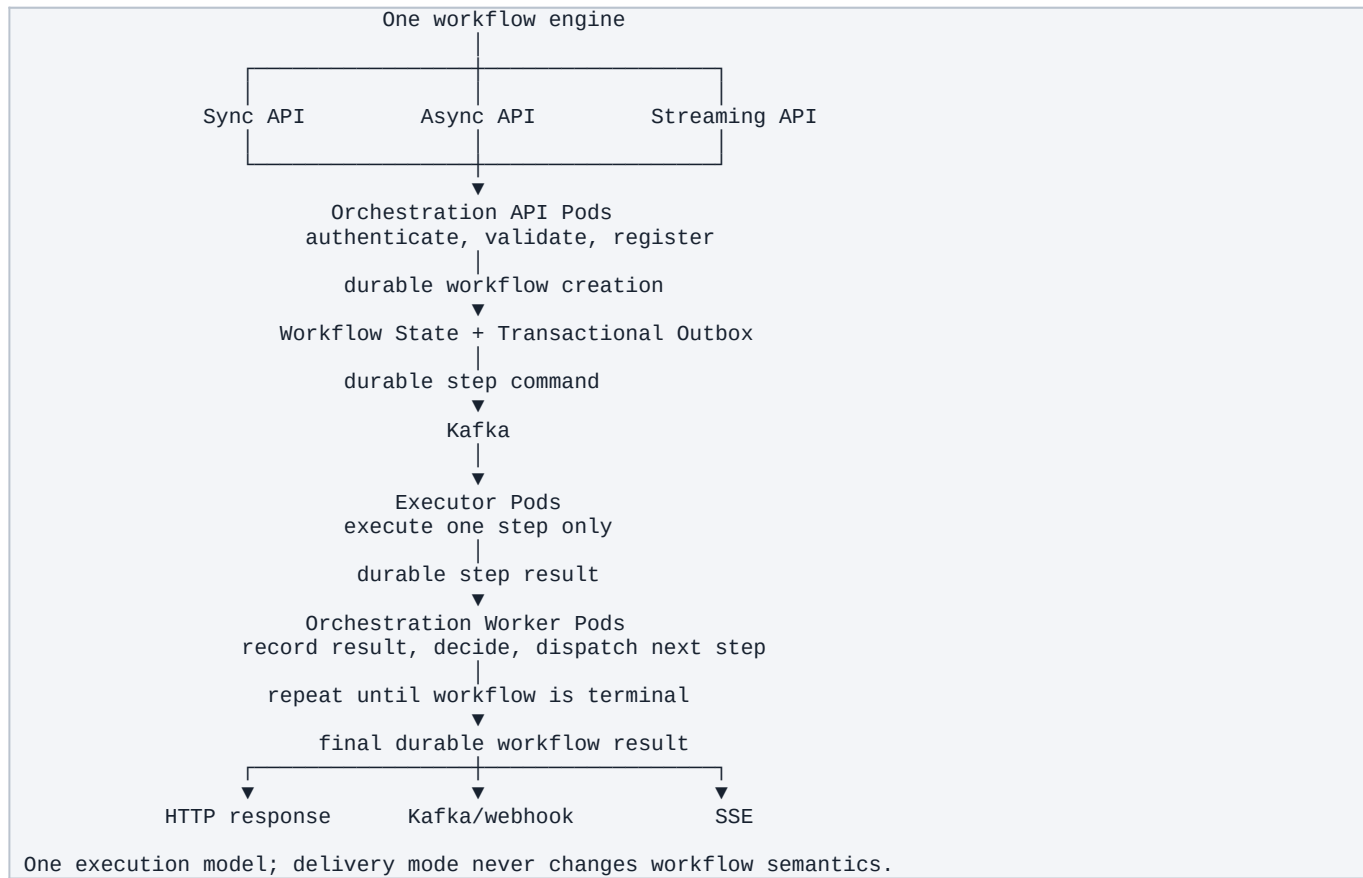


Figure 6 — Recommended target architecture

7.2 Core responsibility change

Capability	Current	Target	Change
Plan creation	Orchestration	Orchestration	Persist plan and plan version.
Next-step decision	Executor	Orchestration worker	Move routing-slip advancement out of Executor.
Step execution	Executor	Executor	One independently retryable command.
Retry policy	Implicit/distributed	Orchestration	Central workflow policy.
Workflow progress	Unavailable	Orchestration	Current and total steps are durable.
Cancellation	Unavailable	Orchestration	Stop dispatching next steps; active cancellation is best effort.
Final response	Executor/ResponseService	Orchestration finalizer	Store before delivery.
Recovery	Weak	Any orchestration worker	Resume from checkpoint and lease.

7.3 Why it is stronger

- One workflow engine serves every delivery mode.
- Orchestration can report step progress, enforce policies and coordinate retries.
- Executors remain disposable and horizontally scalable.
- Kafka retains durability, buffering and workload distribution.
- Workflow state can be resumed by a different Orchestration Worker pod.

8. Comparative feature and trade-off analysis

Feature	Current async	A: separate full sync	B: hybrid Kafka tap	C: hybrid result store	Recommended true orchestration
Execution engines	One routing-slip engine	Two engines	One routing-slip engine	One routing-slip engine	One durable orchestrated engine
Workflow owner	Executor/routing slip	Sync: Orchestration; Async: Executor	Executor/routing slip	Executor/routing slip	Orchestration
Caller modes	Async only	Async + sync	Async + sync	Async + sync	Sync + async + stream
Kafka internal	Yes	Async only	Yes	Yes	Yes, per-step
Backpressure	Strong	Weak on sync path	Strong	Strong	Strong
Multi-agent support	Yes	Requires second implementation	Yes unchanged	Yes unchanged	Yes, centrally controlled
Progress reporting	No plan progress	Sync path only	Limited	Limited	Yes
Cancellation	No stable owner	Possible for sync path	Limited	Limited	Natural between steps
Final response durability	Not guaranteed as API result	Usually connection-bound	Event/registry dependent	Yes	Yes
Pod failure recovery	Weak for in-flight work	Weak unless checkpointed	Execution continues; connection may drop	Execution continues; result retrievable	Workflow resumes from checkpoint
Read amplification	Normal group consumption	None for sync	N orchestration pods read events	Low	Normal partitioned groups
API pod affinity	None after acknowledgement	Held sync request	Held sync request	Held only until timeout	Held only for response wait
Executor pod coupling	Low	High on sync path	Low	Low	Low
Implementation blast radius	Baseline	Very high	Low	Low to medium	High but strategic
Cloud-native scaling	Good execution, weak workflow state	Poor to moderate	Moderate	Good interim	Strong
Multi-shard readiness	Low	Low	Moderate	Moderate	High with shard ownership
Best use	Existing Kafka/webhook clients	Short single-step calls only	Fast sync façade	Safe interim sync design	Long-term platform target

8.1 Decision summary

Option	Decision	Rationale
Current async	Retain during migration	Preserves existing clients, but does not meet new sync and workflow-control needs.
Option A	Do not adopt as general platform design	Duplicates orchestration and weakens resilience.
Option B	Acceptable tactical step	Fastest way to prove sync delivery, but fan-in must be controlled.
Option C	Preferred interim step	Durable result store removes the worst fan-in and retry problems.
Recommended target	Adopt strategically	Best consistency, recovery, progress, scaling and shard model.

9. Detailed target-state component design

9.1 Orchestration API deployment

- Terminates HTTP requests for sync, async and stream modes.
- Performs JWT/COIN authorization, validation, configuration lookup and idempotency checks.
- Creates the workflow and plan in PostgreSQL in a short transaction.
- Does not own the workflow for its whole lifetime.
- For sync mode, waits only for terminal state up to a bounded application timeout.
- For long-running workflows, returns 202 with authenticated status and result URLs.

9.2 Orchestration Worker deployment

- Consumes step-result events through a normal Kafka consumer group.
- Claims the workflow transition using optimistic locking and, where required, a short lease.
- Records the result, applies retry/deadline/cancellation policy and chooses the next action.
- Writes the workflow transition and next outbox record in one database transaction.
- Releases ownership after one transition; it does not hold the workflow between steps.

9.3 Outbox Publisher deployment

- Reads unpublished workflow_outbox records.
- Publishes commands and workflow events to Kafka.
- Marks records published only after broker acknowledgement.
- Retries safely without advancing workflow state twice.
- Scales by shard or partition of the outbox table.

9.4 Executor Worker deployment

- Consumes one AGENT_STEP_COMMAND.
- Uses workflow_id + step_id + attempt_id as the idempotency boundary.
- Executes one agent or tool step.
- Writes detailed invocation, LLM, tool and error audit data.
- Persists or outboxes one AGENT_STEP_RESULT.
- Does not identify or publish the next workflow step.

9.5 Delivery Worker deployment

- Reads terminal workflow-delivery records or completion events.
- Sends caller Kafka responses and webhooks without rerunning the workflow.
- Retries delivery independently from execution.
- Stores delivery status, attempt count and last error.
- Supports dead-letter and manual redrive for persistent delivery failures.

9.6 Streaming/SSE Gateway

- Reads platform-owned workflow events rather than caller-owned Kafka topics.
- Converts workflow and AG-UI events into Server-Sent Events.
- Supports reconnect using an event identifier or durable event cursor.
- Is scaled by active connection count and event throughput.
- Does not own workflow execution.

10. API and delivery-mode design

10.1 Endpoint strategy

Mode	Endpoint pattern	Behavior
Existing async	POST .../task-executor	Immediate acknowledgement; final result to Kafka/webhook.
New sync	POST .../task-executor/sync	Wait for terminal result; return 200 or 202.
Optional stream	POST .../task-executor/stream or stream=true	SSE workflow events and final result.
Status	GET .../executions/{x_correlation_id}	Authenticated workflow status and progress.
Result	GET .../executions/{x_correlation_id}/result	Authenticated final response retrieval.
Cancel	POST .../executions/{x_correlation_id}/cancel	Request cancellation and prevent next-step dispatch.

10.2 Synchronous response policy

Workflow finishes before the sync wait limit
COMPLETED → HTTP 200 + final response FAILED → mapped terminal error response
Workflow is still running at the sync wait limit
HTTP 202 Accepted x_correlation_id status_url result_url
The workflow continues independently of the HTTP connection.

Figure 7 — Bounded synchronous waiting

10.3 Idempotency rules

Condition	Required behavior
No existing workflow	Create and execute.
Existing RUNNING workflow, same request hash	Continue waiting or return 202 with the same workflow.
Existing COMPLETED workflow, same request hash	Return stored result.
Existing FAILED workflow, same request hash	Return stored terminal failure or an explicit retry policy outcome.
Same correlation ID, different request hash	Return 409 Conflict.

11. Workflow state, Kafka contracts and reliability controls

11.1 Workflow state model

Logical table	Purpose
workflow_execution	Workflow identity, caller, shard, delivery mode, plan, status, deadline, final response, version and lease.
workflow_step	Stable step identity, sequence, agent, status, input/output references and retry policy.
workflow_step_attempt	Attempt identity, executor instance, timing, result and error.
workflow_outbox	Unpublished command/event payloads written in the same transaction as state changes.
processed_event	Consumer/message identity used to ignore duplicate result events.

Logical table	Purpose
execution_delivery	Kafka topic, webhook, sync wait registration, delivery status and attempts.

11.2 Workflow and step statuses

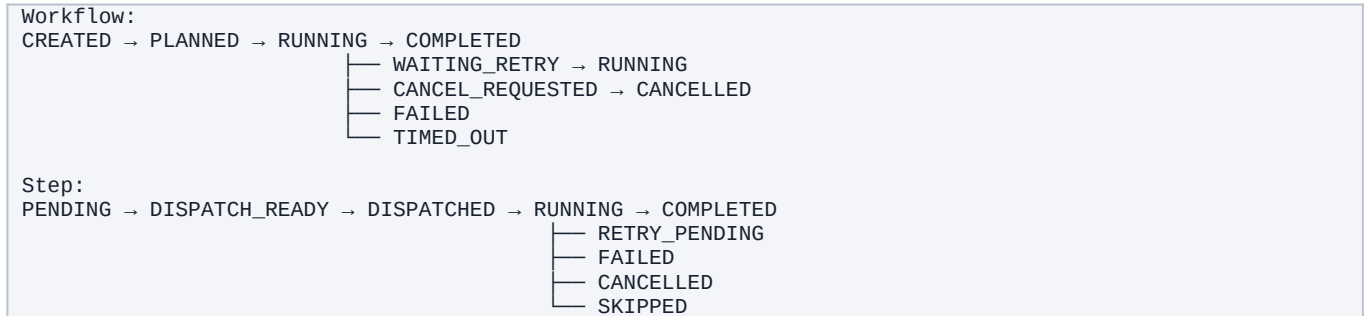


Figure 8 — Proposed state transitions

11.3 Kafka topic model

Topic	Direction	Key	Purpose
agent-step-command	Orchestration → Executor	workflow_id	One executable step command.
agent-step-result	Executor → Orchestration	workflow_id	One terminal attempt result.
workflow-event	Orchestration → delivery/streaming	workflow_id	Progress and terminal workflow events.
workflow-control	Orchestration → Executor	workflow_id	Optional active-step cancel/control.
dead-letter	Any consumer → operations	source key	Messages that cannot be processed automatically.

11.4 Required message identifiers

- workflow_id: internal immutable workflow identity.
- x_correlation_id: caller-visible end-to-end identity.
- shard_id or cell_id: durable workflow home.
- step_id: stable logical step identity.
- attempt_id: one execution attempt for the step.
- message_id: duplicate-event detection.
- schema_version: forward and backward compatibility.

11.5 Reliability patterns

Pattern	Purpose
Transactional outbox	Prevents DB-committed state without a published next-step command.
Idempotent Executor	Prevents duplicate commands from duplicating side effects.
Idempotent result consumer	Prevents duplicate results from advancing the workflow twice.
Manual offset commit	Commit only after state/result durability is secured.
Optimistic locking	Prevents concurrent workers from advancing the same workflow version.
Lease with expiry	Allows another worker to resume after pod failure.
Durable final response	Delivery can be retried without rerunning the workflow.
Dead-letter and redrive	Operational recovery for poison messages and persistent delivery failures.

12. Migration from the current asynchronous architecture

12.1 Migration strategy

The migration should preserve the current asynchronous API contract while gradually moving workflow control from Executor to Orchestration. A dual-mode compatibility period is required so selected use cases can be migrated and rolled back independently.

12.2 Phased roadmap

Phase	Work	Operating state
0. Baseline	Snapshot OpenAPI; measure hop count, duration, Kafka lag, failures and DB connections.	No behavior change.
1. Durable results	Store complete final response and expose authenticated status/result APIs.	Current async engine remains.
2. Sync façade	Add /sync endpoints with bounded wait and 202 fallback.	Option C interim architecture.
3. Workflow schema	Add workflow, step, attempt, outbox and processed-event tables.	Shadow-write current executions where possible.
4. New Kafka contracts	Create step-command, step-result and workflow-event schemas/topics.	No general traffic moved yet.
5. Executor dual mode	Support legacy routing-slip and new single-step commands behind feature flags.	Executor rollback remains available.
6. Orchestration workers	Deploy result consumers, transition engine, outbox publisher and recovery jobs.	Pilot only.
7. Use-case pilot	Route selected low-risk use cases to true orchestration.	Compare response equivalence and reliability.
8. Sync on new engine	Move sync endpoints to the durable workflow engine.	No direct synchronous Executor path.
9. Async migration	Move existing async endpoints to the same engine.	Delivery mode only differs.
10. Progress/cancel/stream	Enable workflow progress, cancellation and SSE.	Orchestration now owns plan state.
11. Retire legacy loop	Disable and remove Executor next-step publishing.	After stabilization and rollback window.

12.3 Component change checklist

Component	Required change
Orchestration API	Add workflow registration, idempotency, sync wait, status/result/cancel APIs.
Orchestration Worker	New deployment for step-result consumption and workflow transitions.
Executor	Add single-step contract; remove next-step decisions after migration.
PostgreSQL	Add durable workflow tables, indexes, retention and migration tooling.
Kafka	Add versioned topics/contracts, manual commit, retry and DLQ strategy.
Delivery	Separate execution completion from webhook/Kafka delivery retries.
Observability	Add workflow traces, step metrics, outbox lag, stuck workflows and shard metrics.
OpenShift/Kubernetes	Add independent deployments, HPA policies, PDBs and graceful draining.

12.4 Compatibility and rollback controls

- Feature flags by usecase_id and caller cohort.

- Executor supports legacy and single-step contracts during migration.
- Existing async response topic/webhook behavior remains byte-compatible until the caller is migrated.
- OpenAPI snapshot tests protect existing endpoints.
- A workflow engine version is stored on every workflow for controlled rollback and audit.
- Dual-run comparison may be used only for side-effect-free workflows.

13. Scalability and performance model

13.1 Independent scaling planes

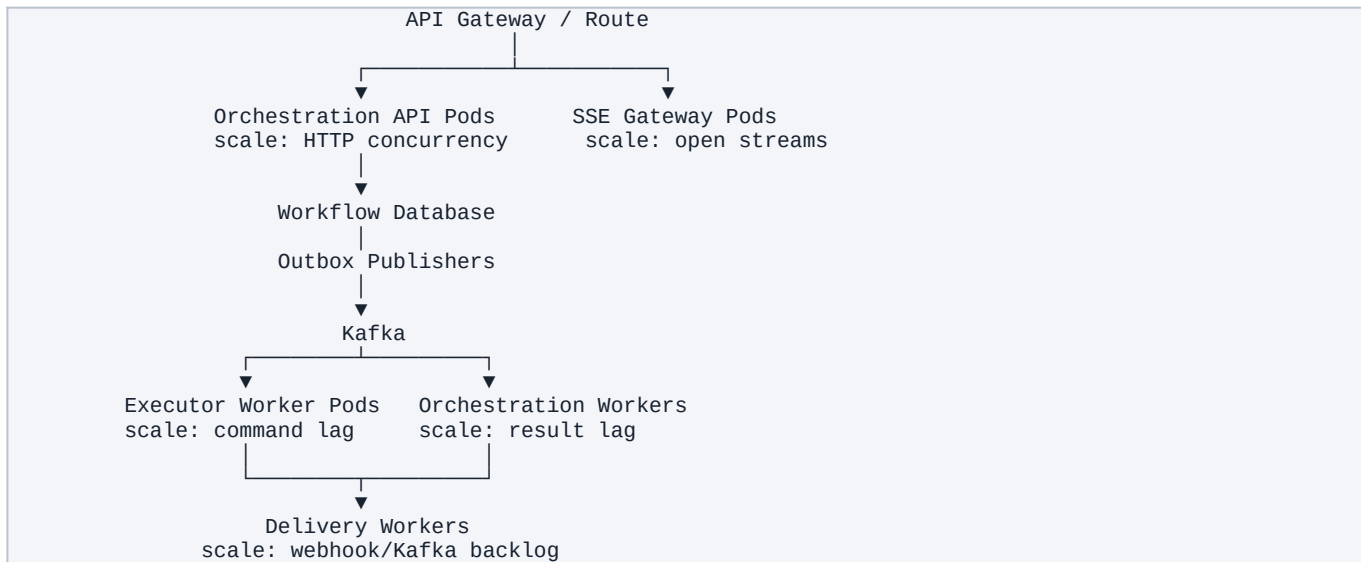


Figure 9 — Independently scalable runtime planes

13.2 Scaling dimensions

Plane	Primary scale signal	Design note
API pods	Request rate, active sync waits, CPU, p95 latency	Do not scale from workflow count alone.
Orchestration workers	Step-result lag, transition latency, outbox backlog	Workers own one transition, not whole workflows.
Executor pods	Command lag, active steps, resource class	Partition count limits useful consumers.
Delivery workers	Webhook/Kafka delivery backlog	Retries do not consume execution capacity.
SSE gateway	Concurrent connections and event throughput	Keep separate from API workers at scale.
PostgreSQL	TPS, locks, IOPS, active connections, table growth	Likely first central scaling constraint.
Kafka	Partition skew, oldest-message age, throughput	Key by workflow_id for ordering and distribution.

13.3 Database connection control

The current pool can open up to 15 connections per pod. As separate API, worker, executor and delivery deployments scale, aggregate connection count can grow faster than request throughput. A database proxy such as PgBouncer or an equivalent managed proxy should be introduced before large pod counts are enabled.

- Use smaller per-process pools and short transactions.
- Separate hot workflow-state rows from large payloads and audit history.
- Index workflow_id, caller/correlation identity, status, lease expiry and unpublished outbox records.

- Partition or archive completed workflow, attempt, event and outbox data.
- Use read replicas for history/reporting, not for workflow-transition decisions that require current state.

13.4 Synchronous connection scaling

- Set an application wait limit below the 900-second route limit.
- Return 202 for workflows that exceed the wait budget.
- Avoid very frequent database polling; use adaptive intervals or a notification optimization.
- Treat the durable workflow row as authoritative even when notifications are used.
- Use graceful draining so routine deployments stop receiving new waits while allowing active waits to finish.

14. Multi-shard and execution-cell architecture

14.1 Shard principle

Shard ownership rule

Every workflow is assigned to one durable home shard or execution cell at creation. Its workflow state, step state, outbox and final result remain in that shard. Many pods may process its steps, but workflow ownership is never split across databases.

14.2 Cell-based target

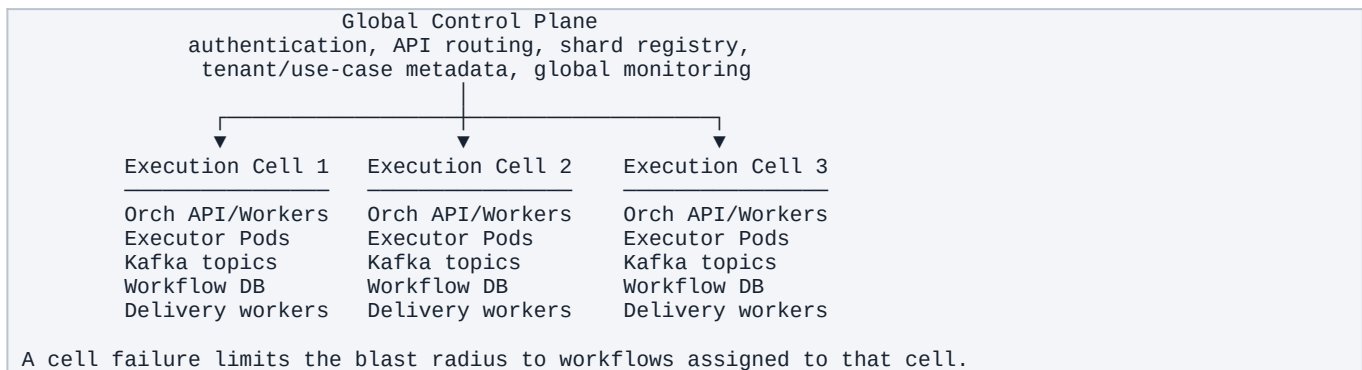


Figure 10 — Future execution-cell architecture

14.3 Shard assignment

Approach	Benefit	Trade-off
Tenant hash	Keeps tenant data together.	Large tenants can create hot shards.
Workflow hash	Usually even distribution.	One tenant spans many shards.
Virtual shards	Maps many logical shards to physical cells and supports rebalancing.	Requires a shard registry.
Dedicated tenant cell	Strong isolation for very large or regulated tenants.	Higher operational cost.

Recommended approach: use virtual shards. The routing layer maps a tenant/workflow to a virtual shard, and the shard registry maps virtual shards to physical execution cells. This supports capacity rebalancing without changing the public correlation identifier.

14.4 Kafka design for shards

- Use `workflow_id` as the Kafka key to preserve ordering within one workflow and distribute different workflows.

- Do not key all messages by shard_id, because that can force a shard onto one hot partition.
- Begin with shared topics carrying shard_id; evolve to topics per cell for stronger isolation.
- Use a separate Kafka cluster per cell only when scale, regulation or region isolation justifies the operational cost.
- Monitor partition skew and oldest-message age by cell and workload class.

14.5 Shard routing for status and result APIs

- Option 1: encode an opaque shard hint in workflow_id so the router can select the cell directly.
- Option 2: maintain a small global directory mapping caller + correlation ID to workflow_id and shard_id.
- The global directory stores routing metadata only, not mutable workflow state.
- A cache can reduce directory reads, but the directory remains the source of routing truth.

14.6 Cross-shard and cross-region rules

- Prefer all workflow control and state in the owning shard.
- Specialized execution may occur remotely, but the result returns to the owning shard for the next decision.
- Do not split ownership of one workflow across multiple databases.
- Assign a home region and keep step-state writes region-local.
- Replicate asynchronously for disaster recovery, audit and analytics rather than synchronously on every step.

15. Cloud-native deployment and operations

15.1 Recommended deployments

Deployment	Responsibility	Primary HPA signal
orchestration-api	HTTP intake, sync waits, status/result/cancel	HTTP concurrency, latency
orchestration-worker	Step-result processing and workflow transitions	Kafka result lag
orchestration-outbox	Publish durable commands/events	Outbox backlog
executor-worker	Execute one agent step	Kafka command lag, active steps
delivery-worker	Webhook and caller Kafka delivery	Delivery backlog
sse-gateway	Long-lived event streams	Open connections, event rate
recovery-scheduler	Expired leases, retry deadlines, stuck workflow scans	Scheduled backlog

15.2 Kubernetes/OpenShift controls

- Readiness fails immediately during shutdown so no new requests or Kafka partitions are assigned.
- terminationGracePeriodSeconds supports active transition completion and connection draining.
- PodDisruptionBudgets protect minimum capacity during node maintenance.
- Separate liveness from readiness; database or Kafka slowness should not create restart loops.
- Kafka-lag-based autoscaling for Executor and Orchestration Worker deployments.
- Resource classes or dedicated executor pools for high-memory, tool-heavy or long-running steps.
- Topology spread constraints and anti-affinity for cell resilience.
- Network policies restrict direct access among API, workers, Kafka and databases.
- Secrets and webhook credentials are referenced securely rather than duplicated in workflow payloads.

15.3 Observability

Area	Required signals
Workflow	Status counts, total duration, stuck age, retry/cancel rate, completion rate.
Step	Duration by agent/tool, attempt count, timeout and duplicate rate.
Kafka	Lag, oldest-message age, partition skew, retry and DLQ volume.
Outbox	Unpublished count, publish delay, retry count and last error.
Database	Connections, lock waits, transaction latency, index hit ratio and partition growth.
Sync API	Active waits, 200/202 ratio, wait duration, disconnects and route timeouts.
Shard/cell	Throughput, saturation, error rate and noisy-tenant concentration.
Tracing	workflow_id, x_correlation_id, shard_id, step_id, attempt_id on every span and log.

16. Failure scenarios and recovery behavior

Failure	Target behavior
API pod dies after workflow creation	Workflow continues. Client retries or reads status/result from another pod.
Orchestration Worker dies during transition	Transaction rolls back or lease expires; another worker resumes from durable state.
Executor dies before completion	Command is redelivered because offset is not committed prematurely.
Executor finishes but result publish fails	Executor outbox republishes the durable result.
Duplicate command	Executor idempotency returns or republishes the stored attempt result.
Duplicate result	processed_event/message_id prevents double workflow advancement.
Kafka unavailable	Outbox records remain pending; workflow state does not falsely advance.
Database unavailable	No transition commits; processing pauses rather than losing workflow position.
Sync wait expires	Return 202; execution continues.
Webhook or caller Kafka unavailable	Delivery retries without rerunning the workflow.
Cancellation before next step	Orchestration marks remaining steps cancelled and does not dispatch them.
Cancellation during active step	Best-effort control signal; no following steps are dispatched.
Cell outage	Only workflows assigned to the cell are affected; recovery follows cell DR policy.

17. Risks, constraints and decision gates

17.1 Main risks

Risk	Mitigation
Database becomes the bottleneck	Pool proxy, short transactions, partitioning, payload separation and cell-local databases.
Duplicate side effects	Attempt idempotency, tool-level idempotency keys and compensating action design.
Kafka partition hotspots	Key by workflow_id, sufficient partitions and workload-specific topics.
Too many workflow writes	Compact state, separate large payloads, archive terminal history and batch non-critical events.
Migration inconsistency	Feature flags, use-case allowlist, response-equivalence tests and

Risk	Mitigation
	controlled rollback.
Long sync connections	Bounded wait, 202 fallback, graceful draining and SSE separated from API workers.
Cross-cell dependency	Keep workflow ownership local and avoid distributed transactions.
Configuration drift	Version configuration and store the version used by each workflow.

17.2 Measurements required before capacity commitment

- Average, p95 and p99 workflow hop count.
- Average, p95 and p99 step duration by execution class.
- Peak concurrent workflows and sync wait connections.
- Kafka bytes and events per workflow, including streaming events.
- Database writes and reads per workflow transition.
- Executor CPU, memory, LLM/tool latency and external rate-limit behavior.
- Current and projected pod counts and aggregate database connection demand.
- Tenant and use-case skew for shard placement.

17.3 Decision gates

Gate	Exit criterion
Gate 1	Durable complete final response is verified and authenticated status/result APIs are available.
Gate 2	New step contracts pass schema, idempotency and failure-recovery tests.
Gate 3	Executor single-step mode matches legacy output for pilot use cases.
Gate 4	Orchestration recovers from pod termination, Kafka rebalance and duplicate events.
Gate 5	Database connection and transition throughput meet projected scale.
Gate 6	Selected use cases meet latency, error and response-equivalence targets.
Gate 7	Async endpoints operate on the new engine before the legacy loop is retired.

18. Final recommendation and end state

18.1 Recommended sequence

1. Introduce durable final-result storage and authenticated status/result APIs.
2. Add synchronous endpoints using the durable result-store approach as the interim architecture.
3. Create workflow, step, attempt, outbox and processed-event persistence.
4. Add versioned step-command and step-result Kafka contracts.
5. Deploy single-step Executor mode and distributed Orchestration Workers behind feature flags.
6. Pilot true orchestration for selected use cases.
7. Move synchronous endpoints, then existing asynchronous endpoints, to the same workflow engine.
8. Enable progress, cancellation and streaming after Orchestration has authoritative plan state.
9. Adopt virtual shards and execution cells as scale and isolation requirements grow.
10. Retire Executor-driven routing-slip advancement only after production stabilization.

18.2 End-state architecture statement

Final architecture

A durable, distributed Orchestration Service owns the plan and workflow state. Executor pods consume one Kafka command and return one durable step result. Any Orchestration Worker pod can advance the workflow from its checkpoint. Sync, async and streaming APIs share this execution engine and differ only in response delivery. Each workflow has one owning shard or execution cell, enabling independent scaling, bounded blast radius and cloud-native multi-pod operation.

18.3 Final flow



Figure 11 — Final end-state flow

Reference basis

This document is based on the supplied current-architecture diagram and the “Supporting Sync and Async Execution — Architecture Decision Document” dated 4 August 2026. Proposed workflow tables, message contracts, outbox, leasing, sharding and execution-cell details extend that baseline into a target-state design.